

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Poster Presentation

Course Code:	DSA0405	Course Title:	FUNDAMENTALS OF DATA SCIENCE
CO Assessed:	CO3 –Poster Presentation	Assessment:	Assessment Tool 1– Poster Presentation
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4)		
Student Name:	Y. Jayesh Ranjan	Reg. No.:	192472201
Section / Batch:	CSE – AI	Date:	27/08/26

Code of Conduct

I Y. Jayesh Ranjan – 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	

Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

DSA0404 – Fundamentals of Data Science
CO5-AT2 Coding Challenge

Q1. Extend your logistic regression implementation to multiclass classification using the Softmax function and categorical cross-entropy (multinomial logistic regression), and separately implement the One-vs-Rest (OvR) strategy using your binary classifier. Compare their decision boundaries on a 3-class 2D dataset.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# 1. Create 3-class 2D dataset
X1 = np.random.randn(50, 2) + [2, 2]
X2 = np.random.randn(50, 2) + [6, 2]
X3 = np.random.randn(50, 2) + [4, 6]

X = np.vstack((X1, X2, X3))
y = np.array([0] * 50 + [1] * 50 + [2] * 50)

X_bias = np.c_[np.ones(X.shape[0]), X]

# 2. Softmax function
def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    exp_z = np.exp(z)
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

# 3. Multinomial Logistic Regression
def softmax_regression(X, y, classes, learning_rate=0.1, epochs=2000):

    weights = np.zeros((X.shape[1], classes))

    y_one_hot = np.zeros((len(y), classes))
    y_one_hot[np.arange(len(y)), y] = 1

    for i in range(epochs):

        scores = X @ weights
        probabilities = softmax(scores)
```

```

    gradient = (X.T @ (probabilities - y_one_hot)) / len(y)

    weights -= learning_rate * gradient

return weights

# 4. Train Softmax model
softmax_weights = softmax_regression(
    X_bias,
    y,
    classes=3
)

# 5. Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))

# 6. Binary Logistic Regression
def binary_logistic_regression(
    X,
    y_binary,
    learning_rate=0.1,
    epochs=2000
):
    weights = np.zeros(X.shape[1])

    for i in range(epochs):
        scores = X @ weights
        probabilities = sigmoid(scores)

        gradient = (X.T @ (probabilities - y_binary)) / len(y_binary)

        weights -= learning_rate * gradient

    return weights

# 7. One-vs-Rest
def one_vs_rest(X, y, classes):

```

```

weights = []

for class_value in range(classes):

    y_binary = (y == class_value).astype(int)

    class_weights = binary_logistic_regression(
        X,
        y_binary
    )

    weights.append(class_weights)

return np.array(weights)

```

```

# 8. Train OvR model
ovr_weights = one_vs_rest(
    X_bias,
    y,
    classes=3
)

```

```

# 9. Prediction functions
def softmax_predict(X, weights):

    probabilities = softmax(X @ weights)

    return np.argmax(probabilities, axis=1)

```

```

def ovr_predict(X, weights):

    probabilities = sigmoid(X @ weights.T)

    return np.argmax(probabilities, axis=1)

```

```

# 10. Predictions
softmax_predictions = softmax_predict(
    X_bias,
    softmax_weights
)

ovr_predictions = ovr_predict(

```

```
    X_bias,  
    ovr_weights  
)
```

```
# 11. Accuracy  
softmax_accuracy = np.mean(  
    softmax_predictions == y  
) * 100
```

```
ovr_accuracy = np.mean(  
    ovr_predictions == y  
) * 100
```

```
# 12. Create mesh grid  
x_min = X[:, 0].min() - 1  
x_max = X[:, 0].max() + 1
```

```
y_min = X[:, 1].min() - 1  
y_max = X[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(  
    np.linspace(x_min, x_max, 300),  
    np.linspace(y_min, y_max, 300)  
)
```

```
grid = np.c_[  
    np.ones(xx.ravel().shape),  
    xx.ravel(),  
    yy.ravel()  
]
```

```
# 13. Grid predictions  
softmax_grid = softmax_predict(  
    grid,  
    softmax_weights  
)
```

```
ovr_grid = ovr_predict(  
    grid,  
    ovr_weights  
)
```

```
softmax_grid = softmax_grid.reshape(xx.shape)
```

```
ovr_grid = ovr_grid.reshape(xx.shape)
```

```
# 14. Plot decision boundaries
```

```
plt.figure(figsize=(14, 6))
```

```
plt.subplot(1, 2, 1)
```

```
plt.contourf(
    xx,
    yy,
    softmax_grid,
    alpha=0.25,
    levels=np.arange(4) - 0.5
)
```

```
plt.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    label="Class 0"
)
```

```
plt.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    label="Class 1"
)
```

```
plt.scatter(
    X[y == 2, 0],
    X[y == 2, 1],
    label="Class 2"
)
```

```
plt.title(
    f'Softmax Multinomial Logistic Regression\n'
    f'Accuracy: {softmax_accuracy:.2f}%'
)
```

```
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
```

```
plt.subplot(1, 2, 2)
```

```

plt.contourf(
    xx,
    yy,
    ovr_grid,
    alpha=0.25,
    levels=np.arange(4) - 0.5
)

plt.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    label="Class 0"
)

plt.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    label="Class 1"
)

plt.scatter(
    X[y == 2, 0],
    X[y == 2, 1],
    label="Class 2"
)

plt.title(
    f'One-vs-Rest Logistic Regression\n'
    f'Accuracy: {ovr_accuracy:.2f}%'
)

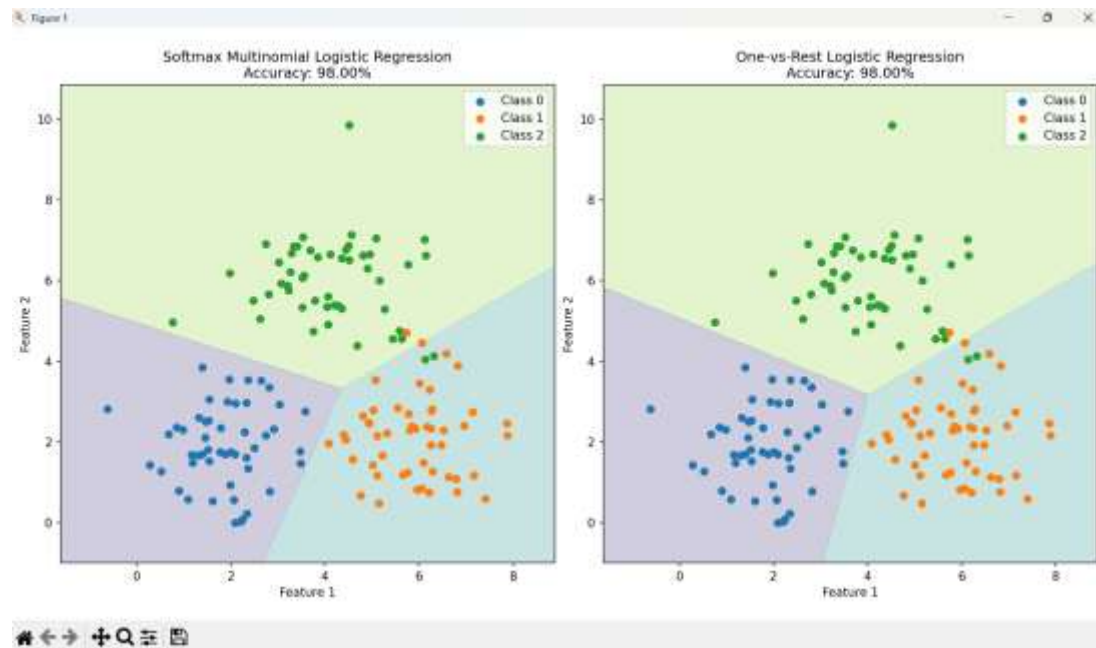
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()

plt.tight_layout()
plt.show()

print("Multiclass Logistic Regression Comparison")
print("-----")
print(f'Softmax Accuracy : {softmax_accuracy:.2f}%')
print(f'OvR Accuracy    : {ovr_accuracy:.2f}%')

```

Output:



Geotag:

